[L0-L1 Core Essence & Design Logic]

L0 Essence: A high-performance columnar analytical database system featuring dense block data layouts with 8192-row sparse index marks, executing query aggregations directly on contiguous columns via CPU SIMD vector loops, and orchestrating log-replicated table merges via ZooKeeper/Keeper consensus.

L1 Logic:
1. Distributed Queries & Protocols (M1) supports native TCP and HTTP interfaces, utilizing two-stage distributed planning to push down query expressions to local shards.
2. Vectorized Execution & SIMD (M2) processes data in column chunks rather than single tuples, utilizing AVX/SSE assembly paths for parallel array processing.
3. MergeTree Storage Engine (M3) groups records by partition key, appending blocks as immutable parts that are asynchronously merged in the background.
4. Sparse Mark & Block Compressions (M4-M6) maps primary key offsets every 8192 rows (Granule) to compressed column bin segments using LZ4/ZSTD and custom codecs.

[L2 ClickHouse Kernel Data Flow Topology]

• [SQL Client Request] → [Distributed Shard Pushdown] → [Partition MinMax Pruning] → [Primary Key Sparse Mark Search] → [Compressed Buffer Read & Block Decompress] → [SIMD Vectorized Filter & Aggregate] → [Coordinator Merge] → [Client Result Stream]

[M1.1] Native TCP & HTTP Protocols (Network Protocol)

• **Dual Protocol Ports**: ClickHouse supports both a high-efficiency binary TCP protocol (default port 9000) and an HTTP REST API (default port 8123) for client connections.
• **Block Serialization**: The binary protocol packs data into columnar chunks (Blocks) before serialization, maximizing network bandwidth utilization during large data transfers.

[M1.2] Two-Stage Distributed Query Execution (Distributed Query Executor)

• **Query Rewrite**: The Distributed engine rewrites incoming queries and projects the execution plan onto remote shards.
• **Parallel Execution**: Remote shards execute local filters and aggregations in parallel. The coordinator node then fetches these intermediate states and merges them into the final result set.

[M1.3] Cluster Shard and Replica Routing (Topology Routing)

• **Shards & Replicas**: The database cluster definition maps shards (for data distribution) and replicas (for high availability).
• **Dynamic Failover**: Distributed writes distribute rows based on a hash of the `sharding_key`. For queries, the coordinator routes tasks to the replica with the lowest response latency.

[M1.4] External Table Engines & FDW (Foreign Data Wrapper)

• **Heterogeneous Access**: ClickHouse supports FDW connectors for `MySQL`, `PostgreSQL`, `ODBC`, and `HDFS`.
• **Clause Pushdown**: The query optimizer analyzes external table schemas and pushes down WHERE filters and LIMIT clauses to minimize remote data transfer.

[M2.1] ColumnVector Contiguous Memory Layout (Column Vector Layout)

• **Contiguous Storage**: Column elements are stored in memory as a flat C++ array (`ColumnVector`).
• **Cache Locality**: Storing values contiguously avoids pointer dereferencing and maximizes L1/L2 cache hit ratios, enabling high-performance memory scans.

[M2.2] Vectorized Loop Chunk Iteration (Vectorized Iteration)

• **Virtual Calls Eliminated**: ClickHouse avoids virtual function calls in inner loops, which slow down traditional row-oriented volcano engines.
• **Chunk Processing**: Data flows between operators in `Chunks` (defaulting to 65,536 rows). Operators process the underlying column vectors using compact, cache-friendly loops.

[M2.3] SIMD Assembly Hard Acceleration (SIMD Optimizations)

• **Register Parallelism**: Loops are compiled using SIMD instructions (SSE4.2, AVX2, AVX512) to process multiple values per clock cycle.
• **Hardware Filter**: A filter operation like `WHERE cost > 100` uses SIMD comparison masks to evaluate multiple columns in parallel, maximizing hardware processing speeds.

[M2.4] Projections & Materialized Views (Projections & MV)

• **Redundant Projections**: `Projections` store table data sorted by different keys to optimize specific query paths.
• **Write Triggers**: Materialized Views act as write-time triggers. When a new part is written, the engine processes the rows and writes them to the view table atomically.

[M2.5] Memory Tracker Query Limitations (Memory Tracker)

• **Granular Allocation**: ClickHouse attaches a `MemoryTracker` to each query execution thread.
• **OOM Prevention**: It tracks memory usage in real-time. If a query exceeds `max_memory_usage`, the tracker aborts the execution to protect the system from out-of-memory crashes.

[M3.1] MergeTree Part Directory & Files (Part Directory Layout)

• **Partition Directories**: Inserts are written as immutable data parts named `PartitionID_MinBlock_MaxBlock_Level`.
• **Column Separation**: Each column has a `.bin` file (compressed values), a `.mrk2` file (sparse index mark pointers), and a global `primary.idx` index.

[M3.2] Background Part Merge Process (Merge Task Pipeline)

• **Part Consolidation**: A background pool selects small, overlapping parts within the same partition.
• **External Merge Sort**: It performs an external merge sort to combine the data into a single, optimized part, removing obsolete source files.

[M3.3] Row/Column TTL Physical Purging (TTL Cleaning)

• **Data Expiration**: Supports TTL configurations at the table, column, and row levels.
• **Merge-Time Cleanup**: Expiration metrics are recorded in `ttl.txt`. During merges, expired rows are physically removed and expired columns are reset to default values.

[M3.4] Mutation Asynchronous Updates (Mutations)

• **Non-Transactional Alterations**: UPDATE and DELETE queries are executed as asynchronous `Mutations`.
• **Part Rewrite**: Since ClickHouse does not support row locks, it copies the source part directory, filters out deleted or updated rows, and writes a new part.

[🏚] ClickHouse OLAP Storage & Execution Trade-offs & Fallacies Matrix

\textbf**Primary keys should be highly specific and include as many columns as possible to speed up multi-criteria searches.:** \textbfClickHouse uses a sparse index. Including too many high-cardinality columns in the primary key inflates the memory size of `primary.idx` and reduces search efficiency by scanning entire granules unnecessarily. → Only include low-to-medium cardinality columns that are frequently used in WHERE filters and are physically ordered. Use secondary Skip Indexes or Materialized Views to accelerate other query patterns.
\textbf**Apply high-concurrency, single-row INSERTs to keep the database updated in real-time.:** \textbfEvery insert creates a new physical part directory. High-frequency inserts overwhelm the background merge process, leading to a part count explosion and triggering the `Too many parts` write throttle error. → Buffer and batch inserts on the client side. Ensure each insert batch contains at least 20,000 rows, or limit inserts to no more than once per second to give background merge workers ample processing time.
\textbf**Execute frequent UPDATE and DELETE statements to modify record states in real-time.:** \textbfUPDATE and DELETE operations in ClickHouse trigger background mutations that rewrite entire data parts. This process causes significant disk I/O and CPU overhead, temporarily degrading query performance. → Use `ReplacingMergeTree` or `CollapsingMergeTree` engines. Insert updated rows with a version or sign column, allowing the engine to deduplicate data during background merges or logic filters.

[M4.1] Primary Sparse Index & Granules (Primary Sparse Index)

- **Sparse Index Sampling**: The `primary.idx` file samples key values at intervals defined by `index_granularity` (default: 8192 rows), rather than indexing every row.
- **Memory Resident**: This sparse layout keeps the index small (typically a few megabytes), allowing it to remain resident in memory for fast lookups.

[M4.2] Mark Files Pointer Mapping (.mrk Files)

- **Index to Bin Map**: The `.mrk2` files map sparse index marks to physical byte offsets in the `.bin` data files.
- **Decompression Boundary**: Each mark entry records the byte offset of the compressed block in the `.bin` file and the offset of the decompressed data within that block.

[M4.3] MinMax Partition Pruning (MinMax Pruning)

- **Partition Pruning**: Tables partition data by a partition key. Each part directory includes `minmax_[partition_key].idx`.
- **Early Part Rejection**: Before reading the index, the query optimizer evaluates the MinMax values of each part and discards directories that do not match the query filter.

[M4.4] Secondary Skip Indexes (Skip Indexes)

- **Granule Pruning**: Secondary indexes (e.g., `minmax`, `set`, `bloom_filter`) are built on non-primary columns.
- **Batch Skipping**: Skip indexes aggregate statistics over a range of granules. If a block of granules does not match the search criteria, the engine skips decompression for those rows.

[M5.1] ReplicatedMergeTree Setup (Replicated Engine)

- **ZK Metadata Mapping**: Replicated table engines register table metadata in ZooKeeper at `/clickhouse/tables/[uuid]/`.
- **Schema Consistency**: All replica nodes pull from the same ZK root to keep schemas and partitioning strategies synchronized.

[M5.2] Replica Logging & Synchronization (Replication Log Sync)

- **Write Log Entry**: After writing a new data part, a replica appends a `GET` entry to the ZK `/log/` queue.
- **Watch Mechanism**: Other replicas monitor the `/log/` node and append the new task to their local `replication_queue`.

[M5.3] Replication Queue Processing (Replication Queue)

- **Log Pulling**: Replicas parse their local queue tasks sequentially.
- **HTTP Part Sync**: If a replica needs a part, it downloads the compressed `.bin` files directly from the source replica via HTTP, rather than through ZooKeeper.

[M5.4] Distributed ON CLUSTER DDL (DDLWorker)

- **DDL Registry**: Running `ALTER ... ON CLUSTER` writes the DDL task metadata to the `/query_log/` path in ZooKeeper.
- **Local Execution**: The `DDLWorker` thread on each node detects the task, executes it locally, and writes the status back to ZK for the coordinator to compile.

[M5.5] ClickHouse Keeper Consensus (CH Keeper)

- **Built-in Consensus**: ClickHouse provides an integrated coordinate daemon called `CH Keeper`.
- **Raft Implementation**: Written in C++, CH Keeper uses the Raft protocol. It is fully compatible with ZooKeeper APIs but uses less memory and offers higher throughput.

[M5.6] ZK Connection Failures & Read-Only Status (ZK Connection Loss)

- **Split-Brain Prevention**: If a replica loses its connection to ZooKeeper and its session times out.
- **Write Disabling**: The node automatically downgrades to `Read-only` mode, rejecting inserts to prevent data divergence.

[M6.1] Chunk and Block Formats (Memory Chunk)

- **Data Vectors**: A `Chunk` is the primary memory container used by the query processor, holding column vectors and row counts.
- **Block Headers**: A `Block` wraps a Chunk with metadata (column names, data types, and OIDs) to define schema boundaries.

[M6.2] Compressed Block Structure (Compression Block)

- **Block Chaining**: `.bin` files are divided into sequential `Compressed Blocks`.
- **Header Boundaries**: Each block header contains 1 byte for the compression codec, 4 bytes for the compressed size, and 4 bytes for the decompressed size. The block is the minimum I/O decompression unit.

[M6.3] LZ4 vs. ZSTD Compression Codecs (LZ4 vs ZSTD Codecs)

- **LZ4 (Default)**: Optimized for decompression speed (reaching gigabytes per second), making it ideal for hot query columns.
- **ZSTD**: Offers higher compression ratios, saving up to 30% of disk space. It is recommended for cold data storage, though it consumes more CPU.

[M6.4] Column-Specific Specialized Codecs (Specialized Codecs)

- **DataType Optimization**:
1. DoubleDelta / Gorilla: Optimized for timestamps or sequential IDs; stores differences of differences to reduce space.
2. T64: Transposes integers to strip out redundant high-order zeros, optimizing storage for low-cardinality fields.

[M6.5] Too Many Parts Write Throttling (Too Many Parts Protection)

- **Part Thresholds**: If a partition contains more than `parts_to_delay_insert` (default: 150), writes are throttled. If it exceeds `parts_to_throw_insert` (default: 300), writes fail.
- **Disk Safety**: This mechanism prevents disk thrashing and forces clients to batch write operations.

[贰] Zone T: ClickHouse Diagnostic & Tuning Reference Laboratory

T1 ClickHouse Typical Tuning Parameters
\texttt:— (:—) \\textit:— \\texttt`max_threads (Total Physical CPU Cores)` \\textitThe max number of threads allocated for a single query. Set to maximum to leverage multicore parallel execution. \\texttt`max_memory_usage (70% - 80% of System RAM)` \\textitThe max memory limit for a single query on a single node. Queries exceeding this limit are aborted to protect the system. \\texttt`max_bytes_before_external_group_by (50% of max_memory_usage)` \\textitSpills intermediate GROUP BY data to disk when memory usage exceeds this limit, preventing OOM crashes. \\texttt`background_pool_size(16 - 32 (Scale with write volume))` \\textitThe number of background threads allocated for part merges. Increase this value under heavy write workloads.

T2 Database Maintenance Commands
\textbf ***1. Diagnose Table Partition Parts Status*** \\texttt– Check active/inactive parts, sizes on disk, and row counts to diagnose Too Many Parts issues \\texttttSELECT partition, name, active, disk_name, bytes_on_disk, rows \\texttttFROM system.parts \\texttttWHERE database = 'default' AND table = 'my_table' \\texttttORDER BY partition, name; \\textbf **2. Check Active Slow Queries and Memory Usage** \\texttt– List currently running queries sorted by real-time memory usage to find candidate queries for KILL QUERY \\texttttSELECT query_id, user, elapsed, memory_usage, read_rows, read_bytes, query \\texttttFROM system.processes \\texttttORDER BY memory_usage DESC LIMIT 10;